Chapter 2

Parallel Computer Architecture

2.1 Learning Objectives

By the end of this chapter, students should be able to:

- distinguish instruction-level, data-level, thread-level, and process-level parallelism;
- explain how pipelining, superscalar execution, SIMD, multicore processors, and hardware threads expose parallelism;
- calculate simple memory-hierarchy and NUMA access costs;
- compare cache coherence with memory consistency and describe the purpose of memory fences;
- explain false sharing and propose layout or partitioning remedies;
- compare UMA, NUMA, distributed-memory, and heterogeneous architectures;
- evaluate interconnection networks using degree, diameter, latency, bandwidth, and bisection bandwidth; and
- select an architecture and data placement for a representative parallel workload.

2.2 Why Architecture Matters

Parallel performance is shaped by where computation executes, where data resides, and how information moves between processing elements. Two programs using the same number of cores may perform very differently. One may reuse data from a nearby cache, while the other repeatedly transfers cache lines between sockets or sends messages across a network.

Architecture therefore determines both the available parallelism and the cost of exploiting it. A programmer does not need to design processors, but must understand the boundaries at which operations become slower: from registers to caches, from one core to another, from one socket to another, and finally from one machine to another.

2.3 Levels of Parallelism

Parallelism appears at several levels of a computer system.

Level	Hardware mechanism	Example
Instruction-level	pipeline and superscalar units	overlapping arithmetic and memory instructions
Data-level	SIMD or vector lanes	adding several array elements per instruction
Thread-level	cores and hardware threads	OpenMP or Pthreads workers on one node
Process-level	separate address spaces and nodes	MPI ranks in a cluster
Accelerator-level	CPU plus GPU or another device	offloading a matrix kernel to a GPU

These levels may be combined. An MPI process can use several OpenMP threads, while each thread executes compiler-generated SIMD instructions on a multicore processor.

2.4 Instruction-Level Parallelism

2.4.1 Pipelining

A processor divides instruction execution into stages such as fetch, decode, execute, memory access, and write-back. Different instructions may occupy different stages at the same time. After the pipeline fills, an ideal processor can complete one instruction per cycle even though each instruction takes several stages to pass through the pipeline.

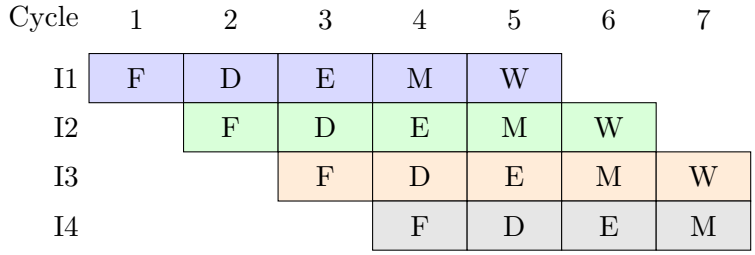


Figure 2.1: A simplified pipeline: F = fetch, D = decode, E = execute, M = memory, and W = write-back.

Dependencies, branches, and cache misses can stall the pipeline. Branch prediction guesses the outcome of a branch so that fetching can continue. A wrong prediction discards speculative work and reduces throughput.

2.4.2 Superscalar and Out-of-Order Execution

A superscalar processor can issue more than one instruction per cycle when suitable functional units and independent instructions are available. Out-of-order execution allows a later independent instruction to proceed while an earlier instruction waits, for example, for data from memory. Results are retired in a controlled order so the program appears to execute correctly.

Instruction-level parallelism is limited by true data dependencies and by the number of independent operations visible within a small instruction window. For large gains, systems also exploit data- and thread-level parallelism.

2.5 Data-Level Parallelism

Single Instruction, Multiple Data (SIMD) units apply one operation to several data elements at once. If a vector register holds eight single-precision values, one vector addition can perform eight additions. Compilers may automatically vectorize loops when iterations are independent and memory accesses are regular.

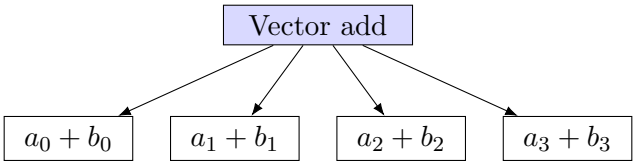


Figure 2.2: SIMD executes the same operation across several lanes.

Vector performance depends on alignment, contiguous access, and minimal branching. SIMD is a CPU mechanism; the much larger SIMT execution model used by GPUs is discussed in Chapter 9.

2.6 Thread-Level Parallelism

2.6.1 Multicore Processors

A multicore processor integrates several cores on one chip. Each core has its own architectural state and can execute a different instruction stream. Cores usually have private L1 caches, may have private or shared L2 caches, and often share a last-level cache and memory controllers.

2.6.2 Simultaneous Multithreading

Simultaneous multithreading (SMT) allows one physical core to maintain the state of multiple hardware threads. When one thread stalls, another may use otherwise idle execution units. Hardware threads share core resources and do not provide the same capacity as additional physical cores.

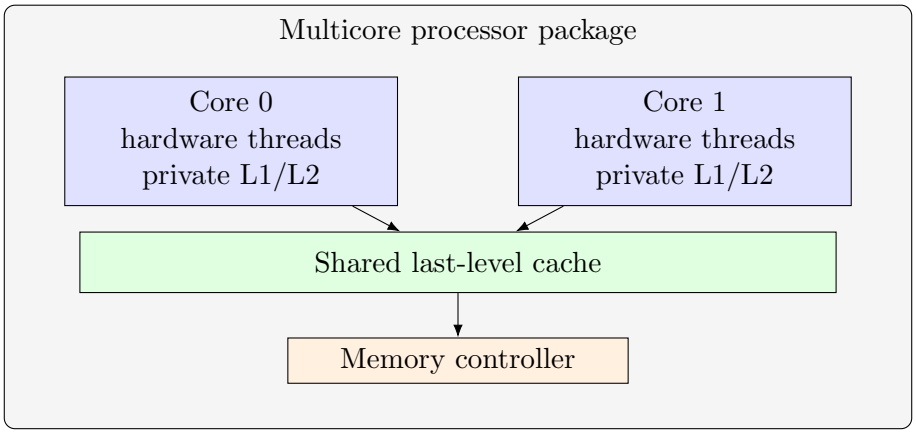


Figure 2.3: A simplified multicore organization with private and shared resources.

2.7 The Memory Hierarchy

Processors execute much faster than main memory can respond. A hierarchy of smaller, faster memories reduces the average access time by keeping recently used data close to the core.

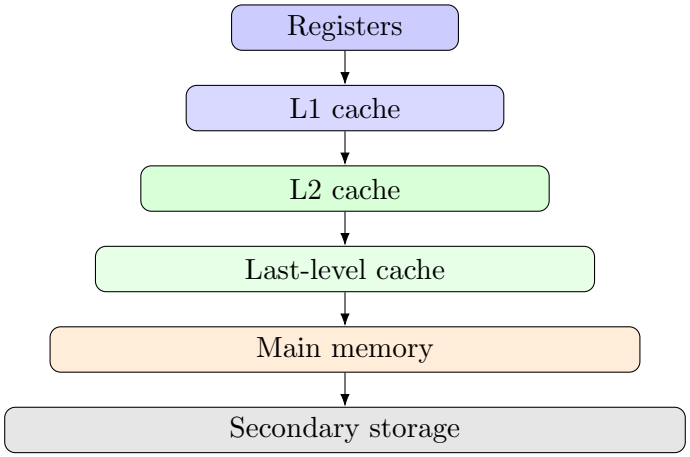


Figure 2.4: Capacity increases down the hierarchy, while access becomes slower.

2.7.1 Cache Lines and Locality

Caches transfer fixed-size blocks called **cache lines**. Accessing one element also fetches nearby bytes. **Temporal locality** means recently used data is likely to be used again. **Spatial locality** means nearby addresses are likely to be accessed soon.

Traversing a row-major matrix by rows typically has good spatial locality. Traversing it by columns may touch a new cache line for each access. Blocking or tiling a matrix computation makes a small region fit in cache so that values are reused before eviction.

2.7.2 Cache Placement and Misses

A direct-mapped cache provides one possible location for each memory block. A set-associative cache provides several possible locations within a set, reducing conflicts at the cost of more complex lookup.

Cache misses can be classified as:

- **compulsory misses**: the first access to a block;
- **capacity misses**: the working set is larger than the cache;
- **conflict misses**: blocks compete for the same cache set; and
- **coherence misses**: another core's write invalidates a local copy.

2.7.3 Worked Example: Cache Mapping

Suppose a cache contains 64 sets, has four ways per set, and stores 64 bytes in each cache line. Its data capacity is

$$64 \text{ sets} \times 4 \text{ ways} \times 64 \text{ bytes} = 16 \text{ KiB}.$$

The lowest six address bits select a byte within a 64-byte line. The next six bits select one of 64 sets. The remaining upper bits form the tag used to identify which memory block occupies a way.

If five frequently used blocks map to the same set, only four can reside there at once. Repeatedly cycling through those five blocks causes conflict misses even when the rest of the cache is almost empty. Changing the data layout, adding padding, or using a more associative cache can reduce the conflict.

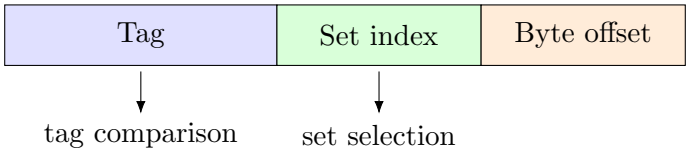


Figure 2.5: A simplified physical address division for cache lookup.

2.7.4 Worked Example: Average Memory Access Time

For a single cache level, a simplified average memory access time is

$$T_{avg} = T_{hit} + r_{miss} T_{penalty},$$

where T_{hit} is cache hit time, r_{miss} is the miss rate, and $T_{penalty}$ is the additional miss cost.

If a cache hit takes 1 ns, the miss rate is 5%, and a miss adds 80 ns, then

$$T_{avg} = 1 + (0.05)(80) = 5 \text{ ns}.$$

Although 95% of accesses hit, the average is five times the hit time. Reducing expensive misses can therefore matter more than reducing arithmetic instructions.

2.7.5 Latency and Bandwidth

Latency is the time until one access or transfer begins to produce a result. Bandwidth is the amount of data transferred per unit time. Pointer-chasing workloads are often latency-sensitive because each address depends on the previous result. Streaming array operations are often bandwidth-sensitive because many independent accesses keep memory channels busy.

2.8 Virtual Memory and Address Translation

Programs use virtual addresses, while memory hardware stores data at physical addresses. The operating system maintains page tables that translate virtual pages into physical frames. Virtual memory provides isolation, protection, controlled sharing, and an address space that is convenient for each process.

Walking a multi-level page table for every load would be expensive. A Translation Lookaside Buffer (TLB) caches recent page translations. A TLB hit allows cache lookup to continue quickly; a TLB miss requires a page-table walk. If the referenced page is not resident in main memory, a page fault may require operating-system action and possibly storage access, which is far more expensive.

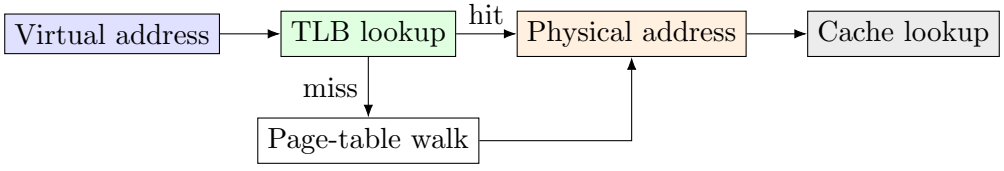


Figure 2.6: The TLB accelerates translation from virtual to physical addresses.

Large pages cover more memory per TLB entry and can reduce TLB misses for large contiguous arrays. They may increase memory fragmentation and should be evaluated rather than assumed to help every workload.

2.9 Memory Channels, Banks, and Controllers

Main memory is organized into channels, modules, ranks, banks, rows, and columns. A memory controller schedules requests and attempts to keep several banks busy. Independent accesses can overlap, increasing throughput, while repeated accesses to the same bank may serialize.

Memory interleaving maps consecutive blocks across channels or banks so streaming workloads can use more bandwidth. The benefit depends on access pattern. A power-of-two stride that repeatedly maps to the same bank may leave other banks idle.

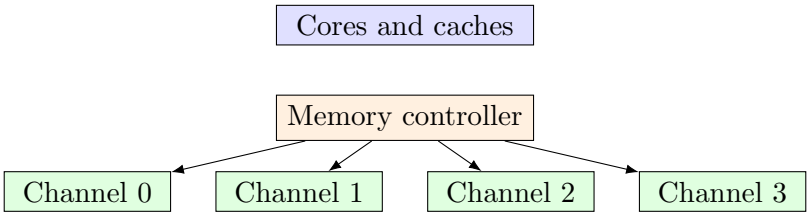


Figure 2.7: Multiple memory channels provide parallel paths to main memory.

2.9.1 Worked Example: Bandwidth Demand

Assume a vector-addition loop reads two 8-byte values and writes one 8-byte result per iteration. Ignoring write allocation and cache reuse, each iteration moves 24 bytes. At 40 GB/s of sustainable memory bandwidth, the bandwidth-only upper bound is

$$\frac{40 \times 10^9 \text{ bytes/s}}{24 \text{ bytes/iteration}} \approx 1.67 \times 10^9 \text{ iterations/s}.$$

Adding arithmetic units cannot exceed this bound unless the program reduces memory traffic or obtains more bandwidth.

2.10 Shared-Memory Architectures

In a shared-memory machine, processors access one logical address space. This does not imply that all memory is equally close.

2.10.1 Uniform Memory Access

In a symmetric multiprocessor with Uniform Memory Access (UMA), processors observe approximately the same main-memory access cost. UMA is straightforward to program, but shared caches, buses, and memory controllers become bottlenecks as the system grows.

2.10.2 Cache-Coherent NUMA

Non-Uniform Memory Access (NUMA) systems attach memory to different processor sockets. A core accesses its local memory through its own controller, while remote memory is reached through an inter-socket link. Cache coherence maintains a single shared address space, so such machines are commonly called ccNUMA systems.

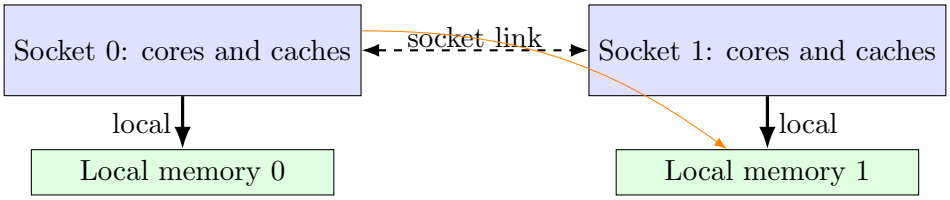


Figure 2.8: In NUMA, placement determines whether memory is local or remote.

Operating systems commonly use **first-touch placement**: a physical page is allocated near the thread that first writes it. Initializing data in parallel with the same partition used for computation can improve locality. Thread affinity keeps workers on chosen cores so their cache and NUMA placement remain stable.

2.10.3 Worked Example: NUMA Placement

Suppose local access takes 90 ns and remote access takes 150 ns.

- With 90% local accesses, the average cost is $(0.9)(90) + (0.1)(150) = 96$ ns.
- With only 50% local accesses, the average cost is $(0.5)(90) + (0.5)(150) = 120$ ns.

The second placement increases average memory latency by 25% without changing the algorithm or processor count.

2.11 Cache Coherence

Private caches may contain copies of the same cache line. If one core writes its copy, other copies must not remain silently usable as though the old value were current. A coherence protocol maintains a consistent value for each individual memory location.

2.11.1 Write-Invalidate and Write-Update

Most general-purpose coherent systems use **write-invalidate**: a writer obtains exclusive ownership and invalidates other copies. In a **write-update** design, new data is sent to other cached copies. Invalidation usually creates less traffic when a core writes the same line repeatedly.

2.11.2 A Simplified MESI Protocol

MESI names four common cache-line states:

- **Modified**: this cache has the only valid copy, which differs from memory;
- **Exclusive**: this cache has the only clean copy;
- **Shared**: several caches may hold clean copies; and
- **Invalid**: the local copy cannot be used.

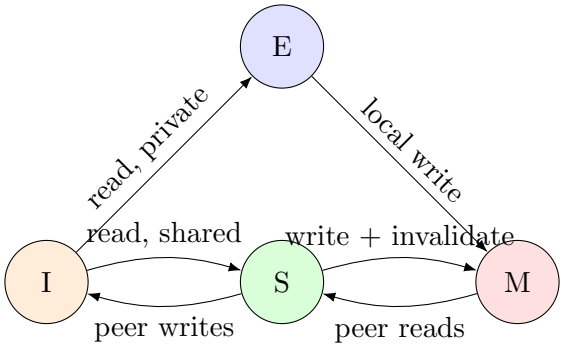


Figure 2.9: Selected transitions in a simplified MESI-style protocol.

Small systems may broadcast coherence requests using **snooping**. Larger systems often use a **directory** that records which caches may hold a line, avoiding broadcast to every processor but requiring directory storage and additional messages.

2.12 False Sharing

False sharing occurs when threads update different variables located in the same cache line. The variables are logically independent, but coherence operates at cache-line granularity. Ownership repeatedly moves between cores, causing invalidations and delay.

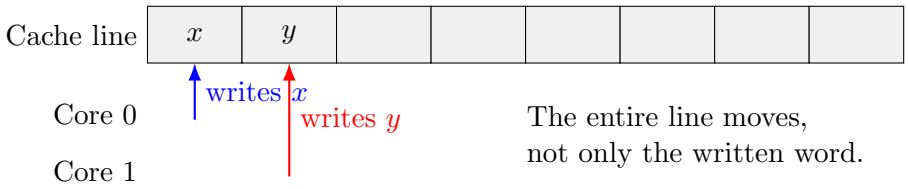


Figure 2.10: Independent variables can interfere when they share one cache line.

Remedies include padding or aligning frequently written per-thread values, storing private values separately, and partitioning data so one thread owns contiguous cache lines. Padding should be used deliberately because it increases memory footprint.

2.13 Memory Consistency

Coherence answers: *what value is eventually observed for one memory location?* Consistency answers: *in what order may reads and writes to different locations appear?*

Under **sequential consistency**, operations appear as if all processors executed in one global order that preserves each processor's program order. Modern processors use relaxed models that permit some reordering to improve performance. Compilers can also reorder operations when the language rules permit it.

Synchronization operations and memory fences constrain ordering. A release operation ensures earlier writes become visible before a synchronization flag is published; an acquire operation ensures later reads observe data made visible by the matching release. These concepts are implemented through language and library primitives in Chapters 5–7. Ordinary timing assumptions or the `volatile` keyword are not substitutes for synchronization.

2.14 Distributed-Memory Architectures

A distributed-memory system contains independent nodes, each with its own processors and memory. A process cannot directly load an ordinary variable from another node. Data is transferred through explicit messages or higher-level communication services.

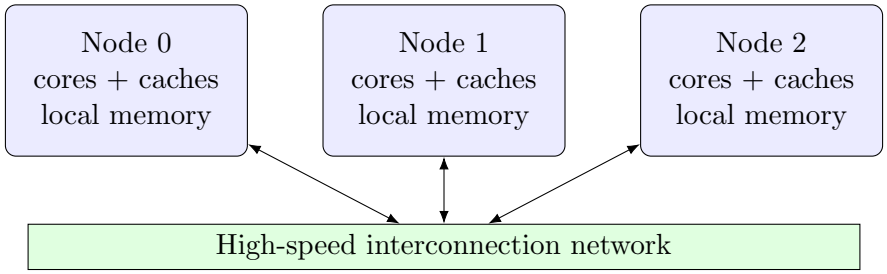


Figure 2.11: A cluster combines shared-memory nodes through a distributed-memory network.

Clusters scale capacity beyond one machine and isolate failures more effectively than one large shared-memory system. Their cost is explicit data distribution and much higher communication latency. Most modern supercomputers are hierarchical: shared memory within a node and distributed memory between nodes.

2.15 Interconnection Networks

An interconnection network links cores, sockets, nodes, memory devices, or accelerators. Important properties include:

- **node degree**: the number of direct links per node;
- **diameter**: the maximum shortest-path distance between two nodes;
- **latency**: the startup and propagation time of a transfer;
- **bandwidth**: sustained data transferred per unit time;
- **contention**: delay caused by transfers competing for links; and
- **bisection bandwidth**: capacity across a cut dividing the network into two similar halves.

2.15.1 On-Chip Interconnects

The same topology questions arise inside a processor. A bus is simple but allows limited simultaneous traffic. A crossbar can connect several requesters and destinations concurrently but becomes expensive as the number of endpoints grows. Rings reduce wiring complexity, while two-dimensional meshes scale to many cores through local links and routing.

An on-chip network carries cache misses, coherence requests, data responses, and memory-controller traffic. A core may therefore slow down because another core generates heavy traffic even when the two threads do not share application data. This is one source of performance variability in multicore systems.

2.15.2 Routing and Contention

Routing determines which path a packet follows. Deterministic dimension-order routing is simple and predictable; adaptive routing may avoid congested links. Flow control divides messages

into smaller units and manages buffer space. Poor routing or insufficient buffering can create head-of-line blocking, where one delayed packet prevents unrelated packets behind it from advancing.

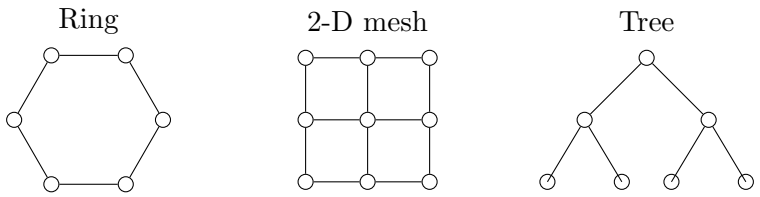


Figure 2.12: Representative network topologies. A torus adds wrap-around links to a mesh.

Topology	Strength	Limitation	Typical use
Bus	simple, low cost	shared bottleneck	small shared systems
Crossbar	many simultaneous paths	cost grows rapidly	switches and memory fabrics
Ring	regular, modest wiring	diameter grows with size	on-chip links
Mesh/Torus	scalable local wiring	multi-hop latency	manycore and HPC networks
Tree/Fat tree	high aggregate bandwidth	expensive upper levels	clusters and data centers

2.15.3 Worked Example: Network Diameter

For a bidirectional ring of N nodes, the diameter is $\lfloor N/2 \rfloor$. A 16-node ring therefore has diameter 8. For a 4×4 two-dimensional mesh, the longest minimal route crosses three horizontal and three vertical links, giving diameter 6. The mesh offers a smaller worst-case hop count but requires more total links.

2.16 Heterogeneous Architectures

Heterogeneous systems combine processors with different strengths. CPUs provide flexible control and strong single-thread performance. GPUs provide high throughput for regular data-parallel work. Other accelerators may target machine learning, signal processing, networking, or encryption.

In a **discrete-memory** design, CPU and accelerator have separate physical memories, so data must be copied across an interconnect. In a **unified-memory** design, devices share an address space or migrate pages automatically, simplifying programming but not eliminating movement cost.

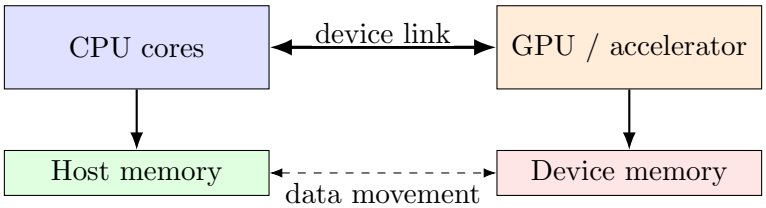


Figure 2.13: A discrete accelerator adds another computation and memory boundary.

Efficient heterogeneous programs assign control-heavy or irregular work to CPUs,

throughput-oriented kernels to accelerators, and keep data near the processor that will reuse it.

2.17 Integrative Case Study: Matrix Multiplication

Consider multiplying dense matrices $C = AB$.

1. **Instruction level:** independent multiply-add operations allow pipelining and out-of-order execution.
2. **SIMD level:** one vector instruction updates several elements of C.
3. **Cache level:** tiling reuses blocks of A and B before they leave cache.
4. **Multicore level:** threads own different output tiles, avoiding write conflicts.
5. **NUMA level:** first-touch initialization places each tile near the socket that computes it.
6. **GPU level:** many threads compute tiles while shared memory reuses input data.
7. **Cluster level:** processes own matrix blocks and exchange panels over the network.

The arithmetic is unchanged at every level. What changes is the placement and movement of data. This is the central architectural principle of parallel computing: computation is often inexpensive compared with moving operands across the memory and communication hierarchy.

2.18 Architecture Selection Checklist

When selecting or analyzing an architecture, ask:

- Is the workload regular or control-heavy?
- How much independent work is available at each level?
- What is the working-set size and access pattern?
- Can data be partitioned so that one processor owns each region?
- Is the workload sensitive to latency, bandwidth, or both?
- How frequently must processors communicate or synchronize?
- Does the problem require capacity beyond one shared-memory node?
- Is the cost of moving data to an accelerator justified by data reuse?

2.19 Summary

- Pipelining, superscalar execution, SIMD, cores, and hardware threads expose different forms of parallelism.
- Caches reduce average access time by exploiting temporal and spatial locality.
- Cache misses and memory bandwidth can dominate arithmetic execution time.
- UMA provides similar memory cost, while NUMA rewards thread and data placement.

- Coherence manages cached copies of individual locations; consistency constrains the apparent order of operations.
- False sharing creates coherence traffic between logically independent variables.
- Distributed-memory systems scale across nodes but require explicit communication.
- Network topology determines hop count, contention, and aggregate communication capacity.
- Heterogeneous systems perform best when tasks and data are placed near the most suitable processor.

2.20 Exercises

1. Distinguish instruction-level, data-level, thread-level, and process-level parallelism. Give one hardware example of each.
2. A five-stage pipeline executes 20 independent instructions. Ignoring hazards, how many cycles are required if the pipeline completes one instruction per cycle after filling? Compare this with an unpipelined processor requiring five cycles per instruction.
3. A SIMD unit has eight lanes. What is the maximum number of single-precision additions performed by 12 vector instructions? State one reason the observed speedup may be lower than eight.
4. A cache hit takes 2 ns, the miss rate is 4%, and the additional miss penalty is 100 ns. Calculate the average memory access time.
5. A cache has 128 sets, eight ways, and 64-byte lines. Calculate its data capacity and identify the number of set-index and byte-offset bits.
6. Classify each event as a compulsory, capacity, conflict, or coherence miss: (a) first access to a new array, (b) a working set larger than cache, (c) two addresses repeatedly mapping to one set, and (d) another core invalidating the line.
7. Explain how a TLB miss differs from a page fault. Why can large pages help an array-intensive program?
8. A loop transfers 32 bytes per iteration and the machine sustains 64 GB/s. Calculate the bandwidth-only upper bound on iterations per second.
9. On a NUMA system, local access takes 80 ns and remote access takes 140 ns. Calculate average access time when 75% of accesses are local. How much does the average improve if first-touch placement raises locality to 95%?
10. Two threads update separate counters stored in one cache line. Explain the coherence traffic and propose two remedies.
11. Explain the difference between cache coherence and memory consistency. Why does a coherent system still require locks or atomic operations?
12. Draw a simplified MESI sequence when Core 0 reads a new line, Core 1 reads it, and Core 0 then writes it.
13. Calculate the diameter of (a) a 20-node bidirectional ring and (b) a 5×4 mesh. Which has the smaller worst-case hop count?

14. Compare a snooping protocol with a directory protocol for a 128-core system.

15. Recommend an architecture and data placement for each workload: (a) a latency-sensitive web service, (b) a dense neural-network training job, and (c) a weather simulation exceeding one node's memory.

16. For tiled matrix multiplication, explain how tiling helps caches, NUMA placement, GPU shared memory, and cluster communication.